

A WAKKAQT FIELD DOCUMENT

THE UNOFFICIAL PARTIAL COMPENDIUM

Architecture Expanded Edition

"A single-process desktop application with five layers, seven moods, three serious background jobs, two FFmpeg personalities, one neural network, and enough atomic flags to make cancellation feel like a constitutional right."

COVERS VERSION 2.9.6

An English-language architectural companion for people who want to know not only *what* the code does, but why the pieces are arranged this way - and which piece to blame when the webcam develops opinions.

“No frameworks du jour. No microservices. No ‘let’s rewrite it in Rust’. Just C++17, Qt 6, FFmpeg, FFTW, an ONNX model, and a genuinely startling number of file renames that all have to happen in the correct emotional order.”

Preface - What This Expanded Edition Adds

The original code companion follows the user manual chapter by chapter and explains the machinery behind each screen. This expanded edition keeps that friendly route through the application, but zooms further out and further in at the same time.

It adds four things:

1. **A system-level map** - what kind of architecture WakkaQt actually has.
2. **Runtime diagrams** - who owns whom, which thread does what, and where data changes shape.
3. **Cross-cutting design explanations** - cancellation, atomic file commits, workspaces, state transitions, and build-time fallbacks.
4. **Change maps** - where a future developer should put a new effect, codec, job, session field, or source of chaos.

LAYPERSON'S SUMMARY The short version is that WakkaQt is one desktop programme, not a collection of services. The window tells specialised background workers what to do. Those workers call audio, video, DSP, AI, and filesystem components. Results come back through Qt signals. Files are written somewhere safe before the programme dares replace anything valuable. It is less “cloud architecture” and more “a well-organised recording studio where every cable is labelled because somebody once spent three hours debugging the wrong microphone”.

Table of Contents

- **00 - The Architecture in One Page**
- **01 - The Static Architecture: Source Tree, Libraries, and Dependency Direction**

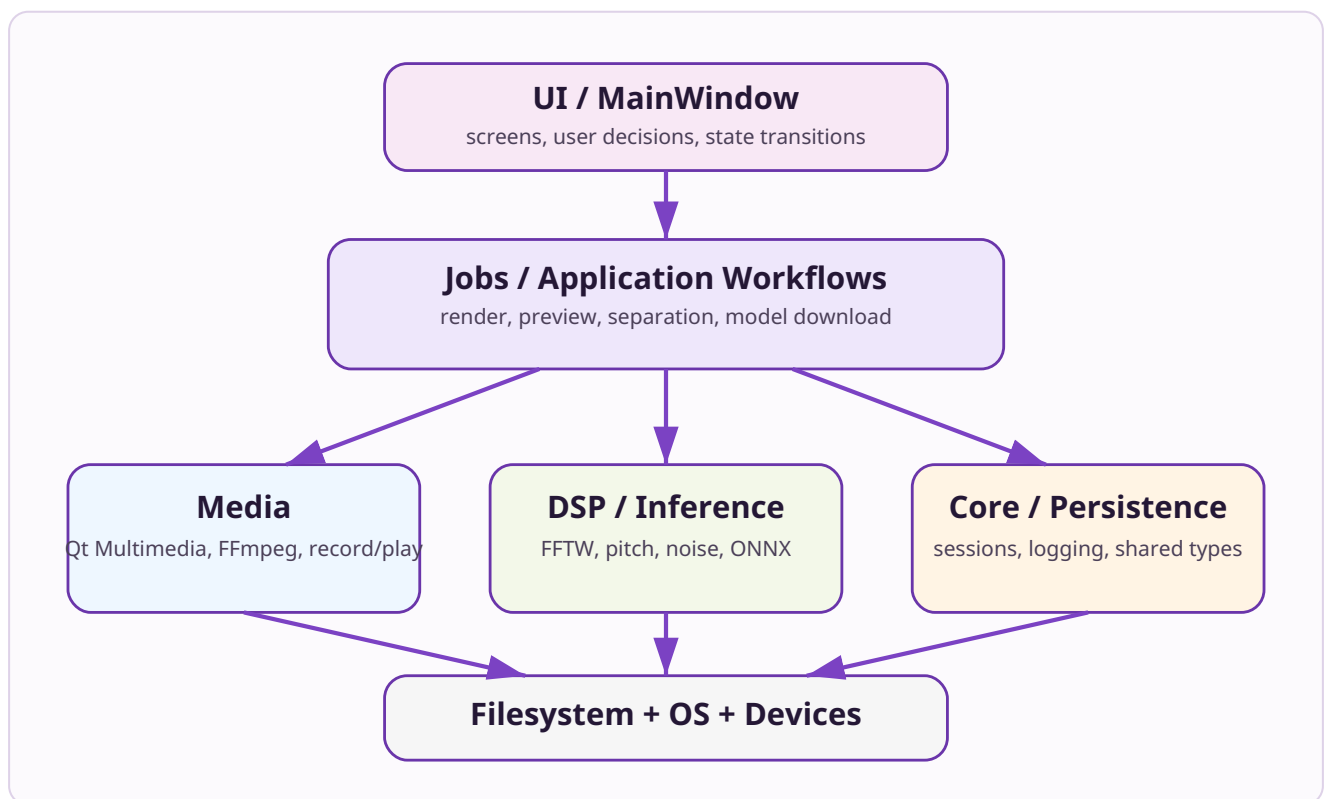
- **02 - The Runtime Architecture: UI Thread, Jobs, Workers, Processes, and Files**
- **03 - Startup: `main()` as the Composition Root**
- **04 - MainWindow: Application Coordinator and Keeper of the Seven Moods**
- **05 - Device and Capture Architecture**
- **06 - Playback, Recording, and Synchronisation Data Flow**
- **07 - Preview Architecture: Normalisation, DSP, and Responsive Rebuilds**
- **08 - DSP Architecture: Pitch, Noise, Formants, Reverb, and Mastering**
- **09 - Video Effects as Data, Not Twelve Bespoke Dialogues**
- **10 - Backing Track Architecture: Model Download, Separation, and Recoverable Export**
- **11 - Rendering Architecture: Native FFmpeg and CLI Fallback Behind One Job**
- **12 - Filesystem Architecture: Workspaces, Sidecars, Backups, and Transactional Intent**
- **13 - Session Architecture: Repository, Snapshots, Restore Workspaces, and Re-entry**
- **14 - Cancellation and Lifetime Architecture**
- **15 - Error Semantics, Logging, and User Recovery**
- **16 - Build-Time Architecture and Optional Capabilities**
- **17 - Test Architecture and Injectable Engines**
- **18 - Extension Recipes: Where New Code Should Go**
- **19 - Architectural Trade-offs and Honest Limitations**
- **20 - Troubleshooting by Layer**
- **Appendix A - Class and Responsibility Atlas**
- **Appendix B - Architectural Glossary for Normal Humans**

00 - The Architecture in One Page

WakkaQt is best described as a **layered, event-driven, single-process desktop monolith with explicit background-job boundaries**.

That sentence sounds expensive, so let us unpack it before anyone invoices us:

- **Layered:** the source tree separates core services, DSP, media infrastructure, background jobs, and UI.
- **Event-driven:** Qt signals and slots move completion, progress, errors, and user actions between objects.
- **Single-process:** almost everything lives inside one application process. The exceptions are tools such as `yt-dlp`, `ffmpeg`, and `ffprobe` when the fallback path is used.
- **Desktop monolith:** there is one deployable application, one main window, one state machine, and no network of tiny services arguing over JSON.
- **Background-job boundaries:** expensive work is wrapped by `RenderJob`, `PreviewJob`, `VocalSeparationJob`, and `ModelDownloadJob`, rather than being performed directly inside button handlers.



The most important architectural distinction is between **control flow** and **media data flow**.

Control flow

User clicks and lifecycle decisions move like this:

```
User action
-> MainWindow validates current State
-> MainWindow builds Params
-> Job starts
-> worker reports progress/completion through signals
-> MainWindow updates UI and chooses the next State
```

Media data flow

Audio and video move like this:

```
Device or file
-> decoder / recorder / extractor
-> canonical internal representation
-> DSP or composition
-> temporary output
-> validation
-> atomic-ish commit to final destination
```

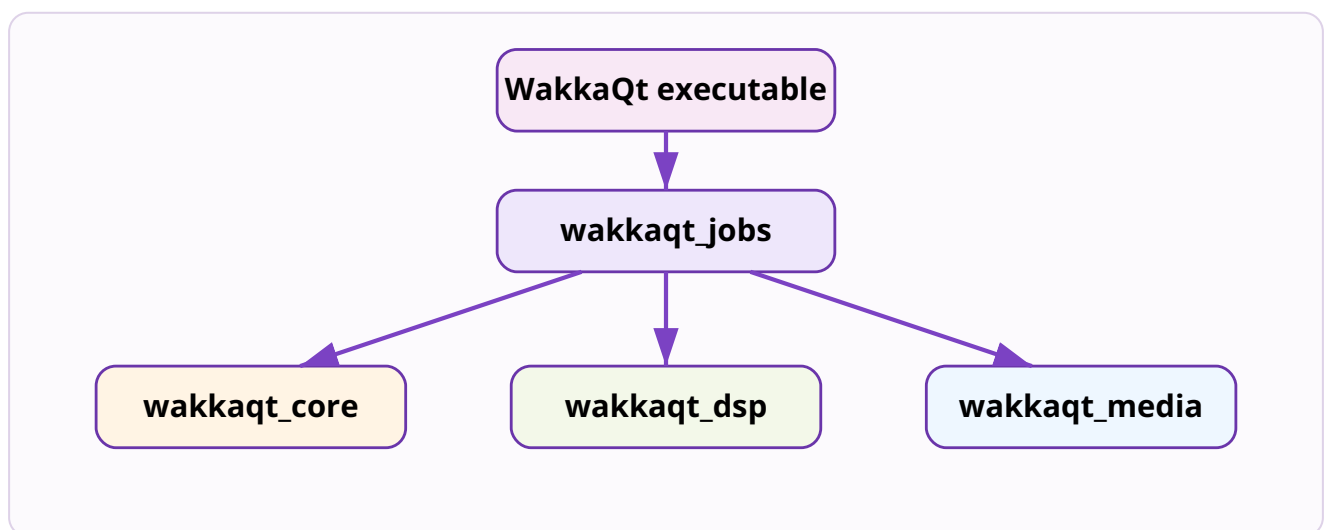
The architecture is deliberately boring in the best sense. There is no framework pretending to be cleverer than the problem. Qt owns the event loop, C++ owns the data, FFmpeg owns most of the unpleasant multimedia truths, FFTW owns the transforms, ONNX Runtime owns the neural network, and the filesystem owns the consequences.

01 - The Static Architecture: Source Tree, Libraries, and Dependency Direction

The source tree is divided into five conceptual areas:

Area	Responsibility	Typical inhabitants
<code>src/core</code>	shared types, persistence, logging, file-oriented infrastructure	<code>SessionRepository</code> , session metadata, common helpers
<code>src/dsp</code>	signal processing and inference-facing algorithms	<code>VocalEnhancer</code> , <code>VocalSeparator</code>
<code>src/media</code>	capture, playback, FFmpeg integration, PCM/WAV handling	<code>AudioRecorder</code> , <code>AudioAmplifier</code> , <code>FFmpegNative</code>
<code>src/jobs</code>	asynchronous use cases with lifecycle and cancellation	<code>RenderJob</code> , <code>PreviewJob</code> , <code>VocalSeparationJob</code> , <code>ModelDownloadJob</code>
<code>src/ui</code>	windows, dialogs, widgets, user orchestration	<code>MainWindow</code> , <code>PreviewDialog</code> , device and library dialogs

At build time, these areas become static libraries rather than independent programmes. This matters: **static-library separation is a compile-time dependency discipline, not a distributed-system architecture.**



The architectural rule behind the folders

Code should depend **downward toward capabilities**, not sideways toward whichever header happens to compile.

- UI may depend on jobs, media, DSP, and core.
- Jobs may depend on core, media, and DSP.
- DSP should not know that a `QPushButton` exists.
- Media should not decide what a dialog says when encoding fails.
- Core persistence should not need a webcam preview widget to save JSON.

The current project is not perfectly pure - very few mature C++ desktop applications are - but the direction is clear enough to support testing and maintenance.

ARCHITECTURAL HONESTY Splitting `MainWindow` across `mainwindowRecorderMgr.cpp`, `mainwindowRenderMgr.cpp`, and friends makes the source readable, but does not create multiple objects. They are still methods of one class sharing one private state. The real separations are the extracted jobs, repository, and media/DSP services. File boundaries are tidy shelves; object boundaries are actual walls.

Why `jobs` is the crucial middle layer

Without the job layer, UI code would have to know:

- how to create worker threads;
- how to stop them;
- how to keep temporary files alive;
- how to avoid double-starting work;
- how to translate native/fallback engines;
- how to report progress;
- how to survive window destruction.

Jobs turn those concerns into a small public interface:

```
start(params);  
cancel();  
isActive();  
waitForFinished();
```

and signals such as:

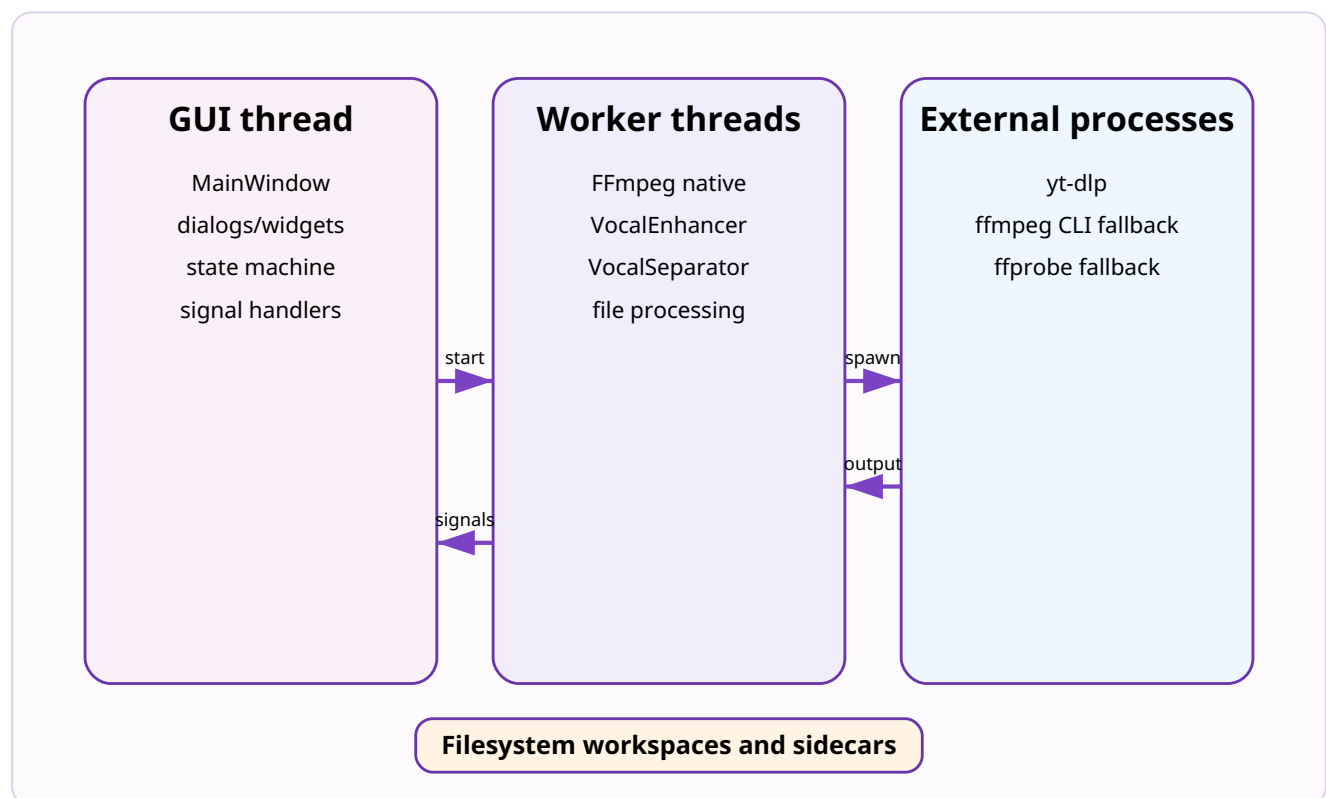
```
progress(double value);  
finished(bool ok, bool cancelled, QString error);
```

This is the closest WakkaQt comes to an **application-service layer**. The UI asks for a use case. The job performs it. The lower layers do the actual maths and media work.

02 - The Runtime Architecture: UI Thread, Jobs, Workers, Processes, and Files

At runtime, WakkaQt has several kinds of execution context:

1. **The Qt GUI thread** - widgets, dialogs, state transitions, and user-facing decisions.
2. **QtConcurrent worker threads** - native rendering, extraction, enhancement, separation, and other expensive functions.
3. **External processes** - `yt-dlp`, fallback `ffmpeg`, and fallback `ffprobe`.
4. **Device-driven callbacks** - audio and video capture through Qt Multimedia.
5. **The filesystem** - not a thread, but frequently the most opinionated participant.



The rule: widgets stay on the GUI thread

Qt widgets are not thread-safe. Background workers therefore do not manipulate progress bars or labels directly. They report information back through queued signals or callbacks scheduled onto the object's thread.

Conceptually:

```
workerThread: progressCb(0.42);
    |
    v
GUI thread: emit progress(42);
           progressBar->setValue(42);
```

QObject context connections also act as a lifetime guard: if the receiving object is destroyed, Qt removes the connection rather than delivering a cheerful callback to a corpse.

Worker ownership

The typical ownership chain is:

```
MainWindow or PreviewDialog
  owns Job QObject
    owns QFutureWatcher / QProcess / timers
      observes worker or external process
```

The worker function receives **copies of paths and parameters**, plus a shared cancellation flag. It should not reach back into mutable widget state while running.

LAYPERSON'S SUMMARY The window is the manager. It is allowed to talk to people but not carry heavy furniture. Jobs are the foremen. Worker threads carry the furniture. External tools are contractors who communicate mostly through stderr and percentages extracted with regular expressions, which is how you know they are contractors.

03 - Startup: `main()` as the Composition Root

`main()` is the composition root: the one place where the operating environment is prepared and the top-level object graph is created.

Its responsibilities are architectural rather than musical:

1. set environment variables required before Qt Multimedia loads;
2. construct `QApplication`;
3. discover optional frei0r plugins;
4. acquire a single-instance lock;
5. install the global Qt message handler;

6. remove old abandoned workspaces;
7. construct and show `MainWindow`.

A representative startup quirk is configured before `QApplication` exists:

```
// Webcam recordings may use a chroma format that hardware decoders reject.  
// Qt reads this setting at plugin-load time, so it must happen first.  
qputenv("QT_FFMPEG_DECODING_HW_DEVICE_TYPES", "");
```

Why startup order matters

Some architectural dependencies are temporal rather than type-based.

- The FFmpeg backend environment must be set before Qt loads the multimedia plugin.
- The log handler must exist before components start producing useful diagnostics.
- Workspace cleanup must happen only after the application knows it is the sole running instance.
- `MainWindow` must be last because its constructor triggers device discovery and builds most of the user-facing object graph.

This is an example of **initialisation architecture**: the code might compile in any order, but the programme only behaves correctly in one order.

Single-instance architecture

WakkaQt uses `QLockFile` rather than inter-process communication. The first process acquires a lock in the temporary directory. A second process fails, displays a message, and exits.

This is intentionally modest. WakkaQt does not forward the second process's command-line arguments to the first. It simply says, in effect:

“One karaoke studio at a time. The neighbours have rights.”

Logging as a cross-cutting service

The global message handler captures `QDebug`, `qWarning`, and `qCritical`, formats them, and forwards them to the on-screen log.

Architecturally, this gives every layer one common diagnostic channel without passing a logger object through every constructor. The trade-off is global state, but for a single-process Qt desktop application, that is a reasonable and very practical bargain.

04 - MainWindow: Application Coordinator and Keeper of the Seven Moods

`MainWindow` is the application's **orchestrator**. It does not perform every expensive operation itself anymore, but it still decides when operations may begin, which files they use, and what the user sees next.

Its code is split across several implementation files:

```
mainwindowDevicesMgr.cpp
mainwindowPlaybackMgr.cpp
mainwindowRecorderMgr.cpp
mainwindowRenderMgr.cpp
mainwindowLibraryMgr.cpp
mainwindowSeparatorMgr.cpp
```

This is a readability technique. The methods still belong to one C++ class.

The seven-state lifecycle

```
enum class State {
    Idle,
    Recording,
    Aborting,
    Finalizing,
    Restoring,
    Rendering,
    Separating
};
```

All transitions pass through one gatekeeper:

```

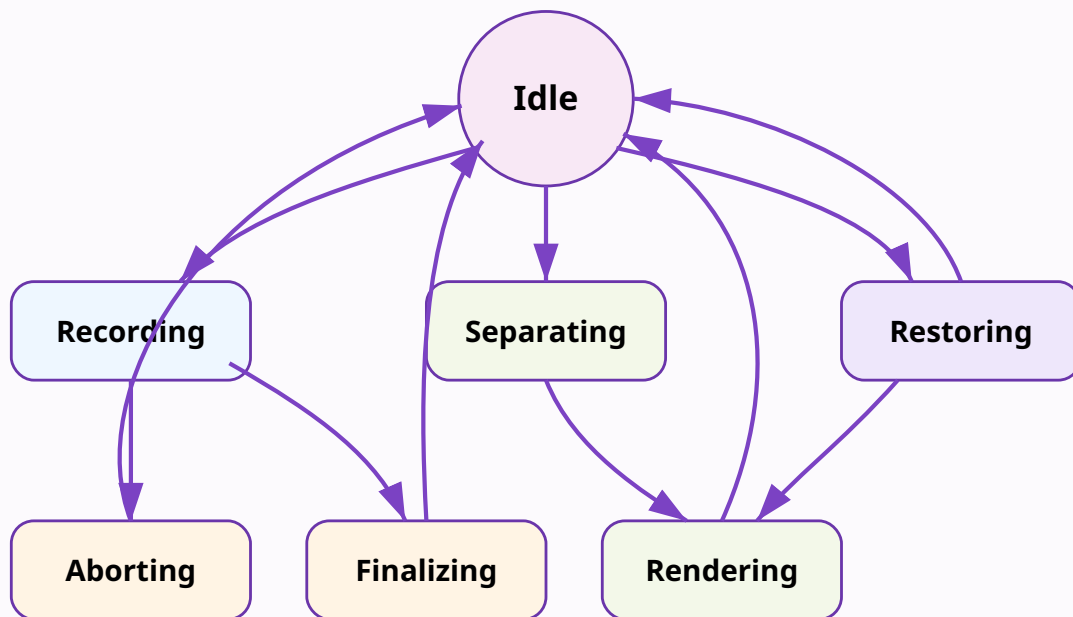
bool MainWindow::trySetState(State next)
{
    if (next == m_state)
        return true;

    static const QMap<State, QSet<State>> allowed = {
        { State::Idle,      { State::Recording, State::Restoring, State::Separating } },
        { State::Recording, { State::Aborting, State::Finalizing, State::Idle } },
        { State::Aborting,  { State::Idle } },
        { State::Finalizing, { State::Rendering, State::Idle } },
        { State::Restoring, { State::Rendering, State::Idle } },
        { State::Rendering, { State::Idle } },
        { State::Separating, { State::Idle } }
    };

    if (!allowed.value(m_state).contains(next)) {
        qWarning() << "Rejected illegal state transition";
        return false;
    }

    m_state = next;
    return true;
}

```



What the state machine prevents

It prevents whole categories of bugs:

- starting separation while recording;
- opening a library restore while rendering;
- beginning a second recording while finalisation still owns temporary files;
- letting each button invent its own definition of “busy”.

What is deliberately not state

Two booleans are data/readiness facts, not lifecycle phases:

```
bool hasCamera;           // hardware available now
bool recordingHasWebcam;  // this recording/session contains video
```

This distinction is subtle and excellent. A restored audio-only session must remain audio-only even if a camera is currently plugged in. Hardware availability does not rewrite history.

MainWindow's remaining architectural role

Even after extracting jobs, `MainWindow` still:

- chooses files and destinations;
- creates job parameter objects;
- owns the application state machine;
- presents recovery choices;
- coordinates live and restored sessions;
- decides when to create or destroy workspaces;
- maps low-level errors into human-facing messages.

That is acceptable: it is the **application controller**. The architectural goal is not to make `MainWindow` tiny at any cost. The goal is to keep long-running algorithms, persistence details, and media mechanics outside it.

05 - Device and Capture Architecture

Device selection is built on Qt Multimedia's discovery APIs:

```
QMediaDevices::audioInputs();
QMediaDevices::videoInputs();
```

The device dialog turns these lists into user choices. The policy is asymmetric:

- microphone missing -> fatal for the recording workflow;
- camera missing -> valid audio-only mode.

```
if (audioInputs.isEmpty()) {
    QMessageBox::critical(this, "Device Error", "No audio devices available.");
    exit(-1);
}

if (videoInputs.isEmpty()) {
    qDebug() << "No camera detected; proceeding audio-only.";
}
```

Capture object graph

A simplified object graph is:

```
MainWindow
|- QMediaCaptureSession
|   |- QCamera (optional)
|   `-- QMediaRecorder (optional video recording)
|
`-- AudioRecorder
    `-- QAudioSource (microphone PCM)
```

Audio recording is handled separately from webcam recording because the vocal pipeline needs predictable raw PCM and WAV control. The camera can remain in Qt Multimedia's video-recording path.

Camera-format scoring

WakkaQt does not blindly accept the first camera format. It prefers formats that avoid unnecessary lossy recompression, with compressed JPEG-style capture used only when needed.

Architecturally, this belongs in the **media-adapter layer**: the UI asks for a camera, but the media component decides which concrete format best satisfies quality and compatibility.

LAYPERSON'S SUMMARY The user chooses “the webcam”. The code chooses “the least troublesome way this webcam can deliver pictures without turning your face into a blocky ransom video”.

Capability degradation

WakkaQt uses graceful capability degradation:

Camera present	-> audio + webcam capture, video preview, video render
Camera absent	-> audio-only capture, audio preview, audio-compatible render
ONNX absent	-> Backing Track button hidden
frei0r absent	-> Vertigo effect hidden
FFmpeg dev absent	-> CLI fallback

This is an architectural pattern: optional capabilities disappear or fall back at the boundary, rather than leaving visible controls that fail later.

06 - Playback, Recording, and Synchronisation Data Flow

The recording workflow combines three clocks and several independent systems:

- playback position;
- microphone capture duration;
- webcam file duration;
- OS/device buffering;
- recorder startup latency.

Pretending all of these begin at exactly the same nanosecond is how synchronisation bugs become family traditions.

Start sequence

The simplified start sequence is intentionally direct:

```
recordingHasWebcam = hasCamera;

vizPlayer->seek(0, true);
player->pause();

audioRecorder->startRecording(audioRecorded);

if (hasCamera && mediaRecorder)
    mediaRecorder->record();

player->play();
```

This starts the recorders immediately before playback. It reduces the initial gap but does not assume the gap is zero.

Stop-time synchronisation

The more reliable synchronisation calculation happens when recording stops. WakkaQt compares actual recorded-file durations with actual playback progress and derives offsets from measured results.

```
playback actually reached P milliseconds
microphone file actually contains A milliseconds
webcam file actually contains V milliseconds

estimated audio offset = function(P, A, capture timing)
estimated video offset = function(P, V, capture timing)
```

The exact formula is less important architecturally than the decision to use **observed artefacts** rather than a parallel stopwatch.

Why offset data is first-class

Offsets survive beyond recording:

- they enter the preview;
- the user can adjust them;
- they are stored in session metadata;
- they are applied again during render;
- restored sessions re-enter the same pipeline with the same timing model.

This makes timing data part of the session's domain state, not a temporary UI tweak.

Streaming WAV architecture

`AudioRecorder` writes a placeholder WAV header, streams PCM behind it, and patches only the size fields when recording ends:

```
// Reserve the standard header with zero-sized payload fields.
writeWavHeader(m_outputFile, m_audioFormat, 0, QByteArray());

// QAudioSource streams PCM after the header.

// On stop: seek back and patch RIFF/data sizes.
```

Benefits:

- constant memory usage during long performances;
- immediate disk streaming;
- no full-file rewrite after recording;
- straightforward recovery and debugging.

The file begins life telling a small lie and ends by updating its paperwork. This is still more honest than many project status reports.

07 - Preview Architecture: Normalisation, DSP, and Responsive Rebuilds

The preview pipeline is a staged operation:

```
recorded vocal file
-> PreviewJob extraction / decoding
-> canonical preview PCM
-> VocalEnhancer processing
-> AudioAmplifier playback with backing track
-> PreviewDialog controls and resynchronisation
```

Canonical audio format

Original sources may be mono, stereo, 16-bit, 24-bit, Int32, or encoded in a container. Before preview components receive them, extraction normalises audio into the internal format the preview pipeline expects.

That is why defensive guards in `AudioAmplifier` are normally unreachable: they protect the invariant rather than define it.

Architecturally:

```
wild outside-world formats
-> extraction boundary
-> predictable internal PCM contract
```

This is a good example of **normalise at the boundary, simplify the interior**.

PreviewJob as a two-stage coordinator

`PreviewJob` wraps extraction and enhancement so the dialog does not run expensive work in the UI thread.

Representative parameter passing:

```
PreviewJob::EnhanceParams params;
params.pitchCorrectionAmount = pitchCorrectionAmount;
params.noiseReductionAmount = noiseReductionAmount;
params.key = selectedKey;
params.scale = selectedScale;
params.reverbMix = reverbMix;

previewJob->enhance(previewInputAudioData, format, params);
```

The parameter object is important. It freezes the user's choices for one run, so the worker does not repeatedly read sliders while another thread is moving them.

Coalescing repeated Apply requests

If enhancement is already running and the user changes sliders again, the dialog can mark a rebuild as pending rather than launching overlapping DSP jobs.

```
Enhancement running
+ user presses Apply again
-> pendingPreviewRebuild = true
-> current job finishes
-> one fresh rebuild starts with latest settings
```

This is a lightweight **coalescing queue**. It prevents ten rapid slider changes from becoming ten concurrent phase vocoders wrestling over the same buffers.

Progress architecture

The DSP engine exposes an approximate progress value. A UI timer polls it and updates the bar.

This is not a precise streaming progress protocol. It is pragmatic observability:

```
QTimer every ~55 ms
-> enhancer.getProgress()
-> progressBar update
```

THE PROGRESS BAR'S LEGAL POSITION It is not lying exactly. It is providing an informed estimate without burdening the inner FFT loops with enough signal emissions to turn signal processing into signal administration.

08 - DSP Architecture: Pitch, Noise, Formants, Reverb, and Mastering

`VocalEnhancer` is a pipeline, not one effect. The order matters because each stage changes the material received by the next.

A conceptual pipeline is:

```
PCM input
-> analysis / pitch tracking
-> noise reduction
-> pitch correction / phase vocoder
-> optional formant preservation
-> tonal or harmonic enhancement
-> dynamics processing
-> reverb
-> limiter / output normalisation
-> PCM output
```

Frequency-domain backbone

FFTW transforms time-domain windows into spectra and back again.

```
samples over time
-> window function
-> FFT
-> magnitudes + phases
-> spectral modifications
-> inverse FFT
-> overlap-add reconstruction
```

Plans and working buffers are created once and reused. This avoids expensive planning and allocation during processing.

Pitch detection vs pitch correction

These are separate responsibilities:

- **pitch detection** estimates the note currently sung;
- **pitch correction** decides the target note and alters the signal.

The real-time monitor uses YIN-style analysis for feedback. The post-recording correction uses phase-vocoder techniques for transformation.

Scale-aware snapping

The correction target is constrained by musical policy:

```
estimated frequency
-> nearest chromatic note
-> filter allowed notes by selected key/scale
-> choose nearest legal target
-> move towards target by correction strength and retune speed
```

This separates **signal analysis** from **musical intent**.

Formant preservation

Pitch shifting changes fundamental frequency. Naively shifting the entire spectrum also moves the vocal tract envelope, producing the famous “chipmunk with contractual obligations” effect.

LPC analysis estimates the spectral envelope so the algorithm can:

1. separate excitation/pitch from vocal-tract shape;
2. pitch-shift the excitation;
3. reapply or preserve the envelope.

Architecturally, this is a reusable sub-pipeline inside enhancement rather than a UI feature. The checkbox merely enables it.

Noise reduction

The spectral subtraction stage estimates a noise floor and suppresses bins that resemble stationary background noise.

The design assumes that some early material is representative of the room's noise. This is a useful compromise for home recording: no separate “capture noise profile” ceremony, because the user already has enough buttons to fear.

Reverb and dynamics

Reverb is a Schroeder/Freeverb-style structure of comb and all-pass filters. Dynamics stages then shape loudness and peaks.

The architectural lesson is that UI sliders should map onto a stable parameter model; algorithms should not know whether a parameter came from a slider, a preset, a test, or a future command-line interface.

09 - Video Effects as Data, Not Twelve Bespoke Dialogues

The effects UI is generated from a shared preset table. Each effect describes:

- display name;
- availability requirements;
- FFmpeg filter expression or builder;
- parameter definitions;
- ranges, defaults, and formatting.

The dialog creates generic controls from those descriptions.

```
videoEffectPresets[]  
  -> generic checkbox  
  -> generic sliders  
  -> effect parameter map  
  -> preview filter chain  
  -> final render filter chain
```

This is a small but valuable **data-driven UI architecture**.

Why it matters

Adding a new effect should mostly mean adding data and a filter builder, not cloning an entire widget panel.

A well-designed new effect follows this path:

1. define preset metadata
2. define parameters
3. define FFmpeg filter fragment
4. optionally define capability check
5. preview and render reuse the same chain

The same chain powers preview and final output, reducing “looked good in preview, rendered like a haunted fax machine” inconsistencies.

10 - Backing Track Architecture: Model Download, Separation, and Recoverable Export

The backing-track feature is actually three architectures joined together:

1. **model lifecycle;**
2. **signal-processing/inference pipeline;**
3. **export lifecycle.**

Model lifecycle

`ModelDownloadJob` handles first-use acquisition:

```
network reply
-> incremental temporary write
-> incremental SHA-256
-> expected checksum comparison
-> safe commit into model cache
```

The model is local and reused. The UI does not call the network from inside the inference engine.

Separation pipeline

Conceptually:

```
input media
-> decode to float PCM
-> STFT
-> MDX-Net ONNX inference on spectral chunks
-> instrumental spectrogram
-> iSTFT
-> instrumental WAV in private workspace
```

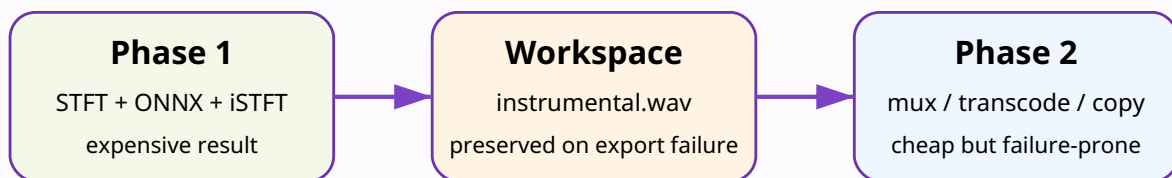
The worker checks a shared atomic cancellation flag during long loops.

Two-phase job design

`VocalSeparationJob` separates expensive inference from comparatively cheap export:

```
// Phase 1
void separate(const QString &inputFile);
void cancelSeparate();

// Phase 2
void exportResult(const ExportParams &params);
void cancelExport();
```



If Phase 2 fails, do not punish the user by repeating Phase 1.

This separation of phases is one of the strongest architectural decisions in the project. Export can fail because of a destination path, codec, disk space, or container. None of those failures invalidate the expensive instrumental already produced.

Recovery UX as architecture

Failure recovery offers actions such as:

- try export again;
- save the instrumental as WAV;
- open the workspace folder;
- discard the result.

This is not just UI polish. It is the user-facing expression of workspace ownership and two-phase processing.

11 - Rendering Architecture: Native FFmpeg and CLI Fallback Behind One Job

`RenderJob` presents one use case with two engines:

```
RenderJob
|- native FFmpeg libraries, in-process
`- ffmpeg CLI fallback, external process
```

The public lifecycle remains the same:

```
start(params);
cancel();
isActive();
waitForFinished();
```

Native path

The native path can:

- decode streams;
- resample and filter audio;
- compose webcam, pitch overlay, and effects;
- encode video/audio;
- report progress from timestamps;
- check cooperative cancellation inside loops.

CLI path

The fallback path:

- builds an FFmpeg command line;
- starts `QProcess`;
- parses stderr for progress;
- terminates or kills the process on cancellation;
- receives completion through Qt process signals.

Compile-time strategy selection

The strategy is chosen by build capability rather than at runtime configuration:

```
#ifdef WAKKAQT_FFmpeg_NATIVE
    startNative(params);
#else
    startFallback(params);
#endif
```

This keeps distribution flexible: systems with development libraries get native integration; others can still use the installed CLI.

Render parameters are a boundary object

`RenderJob::Params` freezes everything required for one render:

- source paths;
- audio/video offsets;
- output path;
- resolution;
- webcam presence;
- effect configuration;
- mix levels;
- related render settings.

A worker should operate on this snapshot, not on live widget values.

Sidecar-first output

Rendering never begins directly on the user's final path.

```
requested: song.mp4
working:   song.partial.mp4
```

The last extension is preserved because FFmpeg uses it to infer the output container.

Only validated success allows the partial to replace the final destination.

LAYPERSON'S SUMMARY The renderer practises on a stunt double. Only when the stunt double survives does the real filename get involved.

12 - Filesystem Architecture: Workspaces, Sidecars, Backups, and Transactional Intent

The filesystem is used as a reliability mechanism, not merely storage.

WakkaQt uses three related patterns:

1. **private workspace directories** for multi-stage operations;
2. **partial sidecar files** for outputs under construction;
3. **backup sidecars** when replacing existing outputs.

Sidecar naming

```
QString sidecarPathFor(const QString &finalPath, const QString &tag);
```

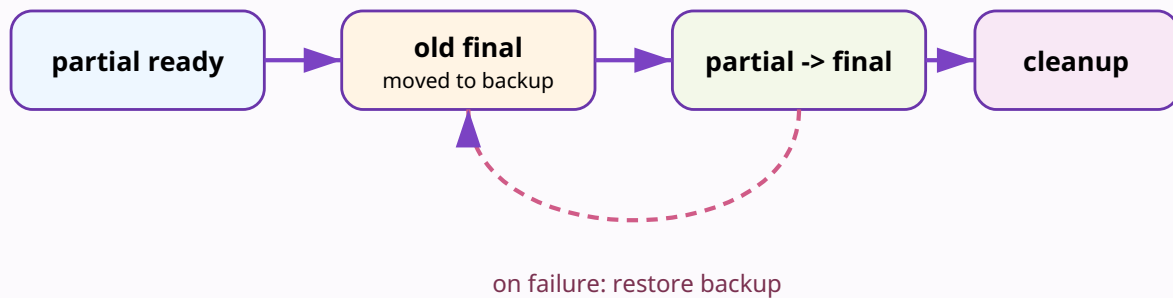
The name must preserve the final extension:

```
correct: song.partial.mp4
wrong:   song.mp4.partial
```

The wrong form asks FFmpeg to identify a mysterious `.partial` container, which is how a robust export feature once briefly became an abstract art project.

Commit sequence

1. produce and validate partial
2. if final exists, move final to backup
3. move partial to final
4. if step 3 fails, restore backup
5. if successful, remove backup



Workspace ownership

A workspace belongs to the job or restore operation that created it.

```
/tmp/WakkaQt_restore_<uuid>/  
/tmp/WakkaQt_separation_<uuid>/
```

This avoids collisions between operations and application instances. The owner decides when the directory can be deleted.

Transactional intent, not database transactions

Filesystem rename semantics differ across platforms and crashes can occur between steps. The design is best described as **transactional intent with rollback for normal failures**.

That is still a major reliability improvement over writing directly to the final destination.

13 - Session Architecture: Repository, Snapshots, Restore Workspaces, and Re-entry

`SessionRepository` is the persistence boundary for recorded performances.

Snapshot model

A session snapshot carries the information needed to save or restore:

- raw vocal path;
- playback path;
- optional webcam path;
- offsets;
- display name/source metadata;
- flags describing available artefacts.

The UI does not manually copy five files and write two JSON documents. It asks the repository to persist a snapshot.

Save architecture

```
validate required artefacts
-> create <uuid>.partial directory
-> copy audio.wav
-> copy playback.wav
-> copy webcam.mkv if present
-> write offsets.json with QSaveFile
-> write session.json with QSaveFile
-> rename .partial directory to final UUID
```

A library entry therefore appears only when complete.

Restore architecture

Restore does not overwrite the live recording temporaries one by one. It creates a private restore workspace and copies validated session artefacts there.

```
library/<uuid>/
-> parse metadata
-> validate WAV and video content
-> copy into restore workspace
-> return RestoreResult with workspace paths
-> MainWindow adopts those paths only after success
```

Re-entry into the live pipeline

After restoration, the application reuses the same preview and render path used after a fresh recording.

```
fresh recording ---->
                        Finalizing / Preview / Render pipeline
restored session --->
```

This avoids maintaining a separate “old session render” implementation that would inevitably become the software equivalent of the forgotten cupboard under the stairs.

Metadata compatibility

`offsets.json` can be absent or invalid. The repository can restore with neutral offsets and warnings, while preferring the session's local `playback.wav` over historical external paths.

This is an example of **backwards-compatible persistence**: be strict about required media, but tolerant about metadata that can safely default.

14 - Cancellation and Lifetime Architecture

Cancellation in WakkaQt is cooperative whenever possible.

Shared atomic flags

A typical worker receives:

```
const std::atomic<bool> *cancelled;
```

Long loops periodically check:

```
if (cancelled && cancelled->load())  
    return false;
```

This pattern is used through decode, STFT, inference, iSTFT, render, mux, and encode paths where the implementation supports it.

Why atomics are used

The GUI thread writes the cancellation request. The worker thread reads it. An atomic provides a cheap, race-free communication flag without locking each processing loop.

External process cancellation

For CLI fallbacks, cooperative cancellation is not always available. The job owns the `QProcess` and can terminate or kill it.

```
native engine -> set atomic flag -> worker exits at safe checkpoint  
CLI engine    -> terminate/kill process -> collect result -> cleanup partial
```

Waiting during shutdown

On application close:

1. active jobs receive cancellation;
2. processes are stopped;
3. watchers are awaited;
4. only then are owning windows destroyed.

This ordering prevents workers from reporting into deleted objects or continuing to write files after application teardown.

QObject ownership

Qt parent-child ownership handles many object lifetimes:

```
new RenderJob(this);  
new QFutureWatcher<bool>(job);  
new QProcess(job);
```

Destroying the parent destroys children. Connections with receiver contexts are disconnected automatically.

IMPORTANT DISTINCTION `QObject` parentage protects object memory. It does not automatically stop arbitrary worker code. That is why jobs still need cancellation tokens and `waitForFinished()`.

15 - Error Semantics, Logging, and User Recovery

A mature media application needs more than `bool` because failure has several meanings:

```
success
cancelled intentionally
failed but result preserved
failed and workspace discarded
restored with warnings
fatal validation failure
```

WakkaQt increasingly models these distinctions through result structures, error strings, and separate signals.

Layered error responsibility

- **media/DSP layer:** detect and describe technical failure;
- **job layer:** classify success, cancellation, and cleanup behaviour;
- **UI layer:** choose a message and recovery options;
- **logger:** preserve diagnostic detail.

For example:

```
FFmpeg cannot mux output
-> media returns failure
-> VocalSeparationJob preserves workspace
-> UI offers Retry / Save WAV / Open Folder / Discard
```

That is a clean separation: the muxer does not open a `QMessageBox`, and the dialog does not know how to preserve ONNX output.

Warnings vs errors

Restoring with neutral offsets is a warning. Missing required audio is an error. Treating both identically would either frighten users unnecessarily or hide real corruption.

Logging architecture

The global Qt message handler provides one timeline across layers. This is especially valuable for asynchronous work, where the visible error may occur after several background stages.

A useful log sequence can tell the story:

```
state Idle -> Separating
workspace created
model checksum valid
STFT complete
inference complete
export started
ffmpeg mux failed
workspace preserved
state Separating -> Idle
```

The log becomes the application's black box recorder, only with more musical-note emojis.

16 - Build-Time Architecture and Optional Capabilities

CMake detects capabilities and defines compile-time features.

Two major examples are:

```
WAKKAQT_FFMPEG_NATIVE
WAKKAQT_ONNX
```

FFmpeg capability matrix

FFmpeg dev libraries available
-> native in-process media engine

FFmpeg dev libraries absent, CLI available
-> subprocess fallback

ONNX capability matrix

ONNX Runtime available
-> compile VocalSeparator and show Backing Track feature

ONNX Runtime absent
-> application still builds; feature is hidden

Optional visual plugins

frei0r is discovered at runtime. Unsupported effects disappear rather than fail on click.

Architectural consequence of `#ifdef`

Compile-time optionality creates multiple supported architectures:

- A. native FFmpeg + ONNX
- B. native FFmpeg without ONNX
- C. CLI FFmpeg + ONNX
- D. CLI FFmpeg without ONNX

Every branch increases testing requirements. The application architecture is therefore not one path with optional buttons; it is a small family of build variants sharing the same UI contracts.

17 - Test Architecture and Injectable Engines

Version 2.9.6 introduces an important testing seam: jobs can receive fake engines.

```
using Engine = std::function<bool(
    const Params &params,
    const QString &partialOutputPath,
    const std::function<void(double)> &progressCb,
    const std::atomic<bool> *cancelled)>;

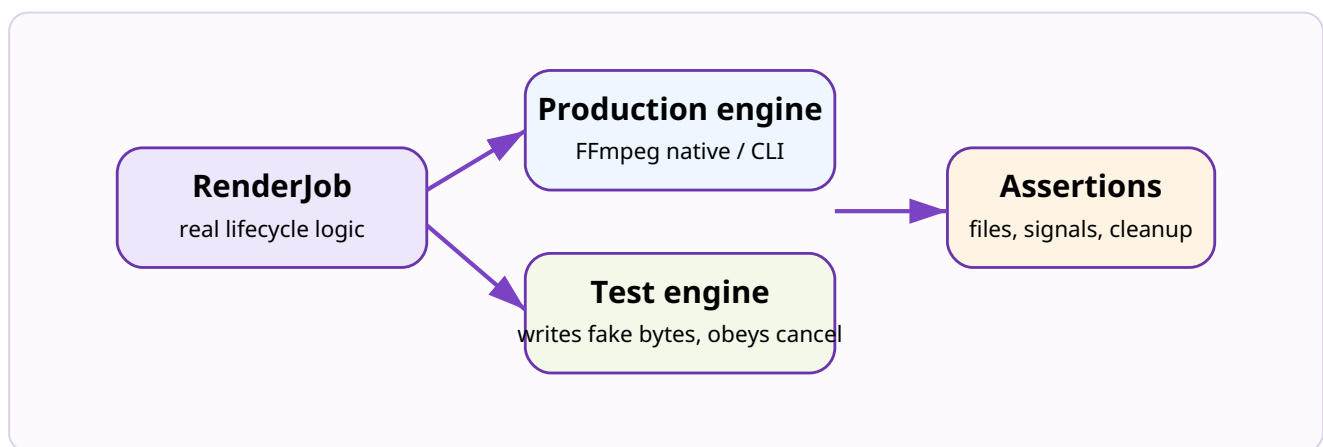
void setEngineForTesting(Engine engine);
```

Why this seam works

The job's responsibilities are:

- lifecycle;
- temporary-path creation;
- cancellation;
- progress forwarding;
- commit/rollback;
- signal emission.

The expensive engine is one call inside that lifecycle. Replacing that one call allows tests to exercise the job's real logic without encoding a video or loading an 80 MB model.



What the tests verify

The suites cover areas such as:

- atomic commit behaviour;
- WAV parsing and malformed chunks;
- session save/restore;
- clamped DSP parameters;
- render cancellation preserving an existing output;
- separation export failure preserving the workspace.

These are architectural tests because they verify contracts between layers, not only arithmetic functions.

CI architecture

The GitHub Actions workflow creates a repeatable build/test environment:

```
checkout
-> install Qt/FFmpeg/FFTW dependencies
-> configure with CMake/Ninja
-> build
-> ctest --output-on-failure
```

It turns “worked on my machine while I was wearing the lucky headphones” into a less persuasive excuse.

18 - Extension Recipes: Where New Code Should Go

Add a new video effect

Put it in the preset data and filter-building path.

```
src/media or shared effect preset definition
-> parameter metadata
-> preview filter chain
-> render filter chain
```

Avoid creating a bespoke dialog unless the effect genuinely cannot fit the generic parameter model.

Add a new long-running operation

Create a job when the operation:

- can take noticeable time;
- needs cancellation;
- creates temporary files;
- has progress;
- must survive UI interaction safely.

Recommended shape:

```
class NewThingJob : public QObject {
    Q_OBJECT
public:
    struct Params { /* immutable input snapshot */ };
    void start(const Params &);
    void cancel();
    bool isActive() const;
    void waitForFinished();

signals:
    void progress(double);
    void finished(Result);
};
```

Add a new session field

1. add it to the snapshot/result model;
2. write it to JSON;
3. read it with a safe default;
4. preserve backward compatibility;
5. decide whether absence is warning or failure;
6. add a round-trip test.

Add a new output format

1. ensure sidecar naming preserves the extension;
2. teach native FFmpeg codec/container selection;
3. teach CLI argument generation;
4. validate partial output;
5. use the shared commit helper;
6. preserve pre-existing destination on failure;
7. test success, failure, and cancellation.

Add a new application lifecycle phase

1. add the enum value;
2. update the allowed-transition table;
3. audit every path that can exit the state;
4. update shutdown behaviour;
5. ensure controls reflect the new busy state;
6. test illegal transitions.

Add an optional dependency

Follow the existing capability pattern:

```
CMake detection
-> compile definition
-> implementation guarded by #ifdef
-> UI capability hidden or fallback selected
-> runtime message only if needed
```

Do not show a shiny button whose main feature is explaining why it cannot work.

19 - Architectural Trade-offs and Honest Limitations

MainWindow remains a large coordinator

The project has extracted real jobs and persistence services, but `MainWindow` still owns broad orchestration. This is not automatically a defect. Desktop applications need a composition/controller layer.

The risk is that future features may add more cross-feature state directly to the window instead of creating a job or service.

Native and CLI paths duplicate behaviour

Supporting both engines improves portability but requires parity in:

- cancellation;
- progress;
- codec choice;
- output validation;
- error messages;
- cleanup.

A bug can exist in only one build variant.

Compile-time options create hidden architecture branches

A developer with all dependencies installed may never exercise the fallback configuration. CI should eventually build more than one capability matrix.

Atomic file commit is not a database transaction

Normal failures can be rolled back. Power loss between filesystem operations still requires recovery policy. The design is strong for a desktop media tool, but the word “atomic” should be understood in context.

Global paths and logging are practical global state

They simplify a single-instance application but make parallel instances and some tests harder. The application already mitigates this through unique workspaces and a single-instance lock.

DSP progress is approximate

Polling a shared progress estimate keeps the inner loops simple. It is responsive enough for users but not mathematically precise.

THE GENERAL PHILOSOPHY WakkaQt repeatedly chooses the smallest architecture that gives a concrete reliability benefit. It does not introduce abstractions merely because conference talks say a modern codebase should contain them. This is why the architecture feels grown rather than imposed.

20 - Troubleshooting by Layer

Symptom	First architectural area to inspect	Why
application exits before window	<code>main.cpp</code> , lock file, environment setup	startup/composition failure
microphone missing	device manager / Qt Multimedia discovery	capture capability boundary
webcam blank but recording exists	startup FFmpeg environment, camera format, preview decode	native media adapter path
SING disabled	MainWindow state and loaded-media readiness	state/control policy
audio/video drift	recorder finalisation and stored offsets	synchronisation data flow
preview freezes after extraction	PreviewJob enhancement/filter phase	worker/cancellation boundary
pitch sounds robotic	VocalEnhancer pitch/formant parameters	DSP stage interaction
backing track button absent	CMake ONNX detection	compile-time capability
model download stalls	ModelDownloadJob/network timeout	network job lifecycle
separation abort is slow	cancellation checkpoints in current phase	cooperative cancellation
render fails only on one machine	native vs CLI engine, encoder availability	build/runtime strategy difference
old output disappears	AtomicFileCommit / caller cleanup	filesystem transaction boundary
session appears but will not open	SessionRepository validation and JSON warnings	persistence boundary
effect previews but does not render	shared filter-chain construction	preview/render parity
tests pass but real encoding fails	FFmpeg engine error propagation	fake engine tests job, not codec internals

Appendix A - Class and Responsibility Atlas

Class/component	Architectural role	Owns/coordinates	Should not do
MainWindow	application controller	state, high-level flows, dialogs, job creation	heavy DSP/encode loops
PreviewDialog	preview UI/controller	sliders, transport, preview state	direct long-running processing
RenderJob	render use case	worker/process, progress, cancel, commit	decide user-facing recovery wording
PreviewJob	extraction/enhancement use case	watchers, canonical PCM result	read live sliders continuously
VocalSeparationJob	two-phase backing-track workflow	separation workspace, export, recovery state	implement neural network maths itself
ModelDownloadJob	model acquisition workflow	network reply, hash, safe write	run inference
SessionRepository	persistence boundary	session folders, JSON, validation, restore workspaces	manipulate widgets
FFmpegNative	native media adapter	decode, filter, encode, mux, inspect streams	show message boxes
AudioRecorder	raw vocal capture	QAudioSource, streaming WAV	decide render options
AudioAmplifier	preview playback adapter	vocal/backing PCM playback	accept arbitrary unnormalised media silently
VocalEnhancer	DSP engine	pitch/noise/formant/reverb/dynamics processing	know about UI controls
VocalSeparator	inference/DSP engine	STFT, ONNX, iSTFT	own export dialogs
YoutubeSearchDialog	search UI adapter	parallel yt-dlp searches, thumbnails	download model or render media
DownloadDialog	download UI/job wrapper	yt-dlp process, progress parsing	parse karaoke pitch
AtomicFileCommit helpers	file commit utility	sidecars, backup, rollback	decide whether output is musically good

Appendix B - Architectural Glossary for Normal Humans

Application controller

The object that decides what the programme should do next. In WakkaQt, this is mostly `MainWindow`, wearing several implementation files like different jackets.

Canonical format

One predictable internal format chosen so every later stage does not need to support every format ever invented by a sound-card manufacturer at 3 a.m.

Composition root

The place where the top-level objects are created and connected. Here, `main()`.

Cooperative cancellation

The worker is asked to stop and checks a flag at safe points. More civilised than deleting the thread and hoping the heap forgives everyone.

Engine injection

Replacing the expensive implementation with a fake function during tests, while keeping the real lifecycle code.

Fallback path

The alternate implementation used when optional native libraries are not present. Usually less elegant, but importantly still invited to the party.

Job

A QObject representing one long-running use case, including start, progress, cancellation, cleanup, and completion.

Layered monolith

One application process whose internal code is arranged into responsibility layers. "Monolith" here means one deployable programme, not "one 80,000-line function called `doEverything()`".

Sidecar file

A temporary file next to the final destination, such as `song.partial.mp4`, produced before the final filename is touched.

Snapshot

An immutable-ish bundle of parameters or session data representing one operation at one moment.

State machine

A list of valid lifecycle states and legal transitions. It prevents the UI from attempting interpretive dance with recording, rendering, and separation simultaneously.

Workspace

A private directory owned by one operation. It keeps intermediate files together and prevents one job from stepping on another job's shoes.

Closing Note

WakkaQt's architecture is not remarkable because it uses exotic patterns. It is remarkable because the patterns are attached to real failure modes:

- the state machine exists because operations must not overlap;
- jobs exist because media work must not block or outlive the UI;
- workspaces exist because expensive intermediate results deserve ownership;
- sidecars exist because final files deserve protection;
- result structures exist because cancellation is not the same as failure;
- test engines exist because job behaviour should be provable without inviting FFmpeg and an 80 MB neural network to every unit test.

The result is a codebase that still feels like a personal, humorous project while behaving increasingly like serious production software.

Or, put less formally:

It is still a karaoke recorder. It has simply developed a state machine, a transactional filesystem strategy, an inference pipeline, multiple execution engines, and enough lifecycle discipline to qualify for a small mortgage.